
Clase 163 — Markov Decision Processes

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Markov Decision Processes + Sutton & Barto cap. 3. Duración estimada: 60 min. · Nivel: Avanzado

Clase 163 — Markov Decision Processes

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Markov Decision Processes + Sutton & Barto cap. 3.
3. Duración estimada: 60 min. · Nivel: Avanzado

Objetivo

Formalizar el marco teórico de RL: un Markov Decision Process (MDP) = tupla (S, A, P, R, γ) . Conocer la Bellman equation que define V y Q óptimos, y los algoritmos clásicos Value Iteration y Policy Iteration que los resuelven (cuando el MDP es conocido).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir un MDP: states, actions, transition prob, reward function, discount.
- Escribir la Bellman equation para V : $V(s) = \max_a \sum P(s'|s,a)[R + \gamma V^*(s')]$.
- Implementar Value Iteration: actualizar iterativamente hasta convergencia.
- Implementar Policy Iteration: alternar policy evaluation + policy improvement.
- Reconocer la limitación: requiere conocer P y R (no aplicable a entornos reales) → motivó model-free (Q-learning, clase 137).

Temas

- Propiedad de Markov: $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t, s_{t-1}, \dots)$.
- Componentes MDP.
- Bellman optimality equation.
- Value Iteration (synchronous update).
- Policy Iteration (alternar eval + improve).
- Convergencia garantizada (contractive operator).

Definiciones y características

- State s : representación del entorno.
- Action a : lo que el agente puede hacer.
- Transition $P(s' | s, a)$: probabilidad de llegar a s' desde s tomando a .
- Reward $R(s, a, s')$: recompensa inmediata.
- Discount γ : peso de recompensas futuras.
- $V^*(s)$: máximo expected return desde s .
- $Q^*(s, a)$: máximo expected return tras tomar a en s .
- Policy óptima: $\pi(s) = \operatorname{argmax}_a Q(s, a)$.

Dataset / recursos

- MDPs sintéticos chicos (FrozenLake, Taxi de Gymnasium).
- Librerías: gymnasium, numpy.

Ejercicios

1. MDP de juguete: definir un MDP de 4 estados con P, R manualmente.
2. Value Iteration: implementar $V[s] = \max_a \sum P(s'|s,a)(R + \gamma V[s'])$ hasta $\max \text{change} < 1e-6$.
3. Policy Iteration: alternar evaluación (V^π) con mejora ($\pi' = \text{greedy}(V)$) hasta estabilidad.
4. FrozenLake: cargar `gym.make('FrozenLake-v1')`, extraer `env.unwrapped.P` (modelo del MDP), resolver con VI.
5. Compare: # iteraciones VI vs PI para llegar a misma policy.

Homework verifiable

Sobre FrozenLake-v1 (`is_slippery=True`):

1. Extraer modelo P y R desde `env.unwrapped.P`.
2. Implementar Value Iteration; reportar V y π greedy.
3. Evaluar la policy con 1000 episodios random; reportar success rate.

Criterio de aceptación: success rate ≥ 0.7 sobre FrozenLake slippery; V^* y policy claramente prefieren caminos seguros.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Value Iteration no converge	$\gamma \geq 1.0$. Fix: $\gamma < 1.0$ (típicamente 0.95-0.99).
Policy Iteration loops infinito	Numerical issues en evaluación. Fix: limit
Acceder a <code>env.P</code> falla	Hay que usar <code>env.unwrapped.P</code> .
Asumir Markov en problemas non-Markov	Por ejemplo, control con velocidad escondi
VI vs PI: cuándo usar?	VI: más simple, más iters pero más baratas

Preguntas frecuentes

¿MDPs en problemas reales?

Casi nunca conocés P y R exactamente. Por eso $\rightarrow$ Q-learning (clase 137) y métodos model-free.

¿POMDPs?

Partially Observable: cuando no ves el state completo. Mucho más difícil. Usar agente con memoria (LSTM, Transformer).

¿Continuous state space?

VI/PI no escalan. Usar function approximation: DQN (clase 137) o policy gradient (135).

¿Bellman equation conexión con DP?

Sí, VI es dynamic programming clásico aplicado a MDPs.

¿Aprender el modelo P, R desde data?

Model-based RL: aprender un modelo, planificar con él. Más sample-efficient pero más complejo.

Referencias

- Géron, cap. 18 — Markov Decision Processes.
- Sutton & Barto, cap. 3-4.
- Bellman (1957), Dynamic Programming.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 164 — TD Learning, Q-Learning, Deep Q-Networks

Apéndice: notebook (primer bloque)

Grid (S=start, H=hole, G=goal): $\begin{matrix} 0 & 1 & 2 & 3 \\ \hline 0 & S & & \\ 1 & & & \\ 2 & & & \\ 3 & & & \end{matrix}$

```
import numpy as np
np.random.seed(0)

nS, nA = 16, 4
holes = {5, 7, 11, 12}
goal = 15

def next_state(s, a):
    row, col = divmod(s, 4)
    if a == 0: col = max(col - 1, 0) # izquierda
    elif a == 1: row = min(row + 1, 3) # abajo
    elif a == 2: col = min(col + 1, 3) # derecha
    elif a == 3: row = max(row - 1, 0) # arriba
    return row * 4 + col

# acciones que ocurren al "resbalar": la intencion + las dos perpendiculares
perp = {0: (0, 1, 3), 1: (1, 0, 2), 2: (2, 1, 3), 3: (3, 0, 2)}

P = np.zeros((nS, nA, nS)) # P[s, a, s']
R = np.zeros((nS, nA, nS)) # R[s, a, s']
for s in range(nS):
    for a in range(nA):
        if s in holes or s == goal:
            P[s, a, s] = 1.0 # estados terminales: absorbentes
            continue
        for a_real in perp[a]:
            s2 = next_state(s, a_real)
            P[s, a, s2] += 1.0 / 3.0
            if s2 == goal:
                R[s, a, s2] = 1.0 # +1 solo al alcanzar la meta
# ... (truncado)
```

Archivos complementarios

- notebook.ipynb